PES-VCS: A Version Control System from Scratch

Name : Shreeranganath Manjunath Saravade
Section : 4K

Introduction :

The PES-VCS project is a simplified version control system inspired by Git. Its objective is to understand how systems track file changes, manage versions, and maintain history using filesystem concepts. In this project, we implemented basic commands like initializing a repository, adding files, creating commits, and viewing logs. The system uses content-addressable storage, where each file is stored using a unique hash. This ensures data integrity and avoids duplication of identical files. Tree structures are used to represent directories, and commit objects maintain the history of changes. Overall, the project provides a clear understanding of how Git-like systems work internally.

Screenshots Section :

Phase 1 : Object Storage Foundation

```
shreeranganath@shreerangnath:~/Desktop/os-u4-orange-problem$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
shreeranganath@shreerangnath:~/Desktop/os-u4-orange-problem$
```

```
shreeranganath@shreerangnath:~/Desktop/os-u4-orange-problem$ find .pes/objects -type f
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
shreeranganath@shreerangnath:~/Desktop/os-u4-orange-problem$
```

Phase 2: Tree Objects

```
shreeranganath@shreerangnath:~/Desktop/os-u4-orange-problem$ make test_tree
make: 'test_tree' is up to date.
shreeranganath@shreerangnath:~/Desktop/os-u4-orange-problem$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

```
shreeranganath@shreerangnath:~/Desktop/os-u4-orange-problem$ xxd .pes/objects/0b/d69098bd9b9cc5934a610ab65da429b52536114
7faa7b5b922919e9a23143d | head -20
00000000: 626c 6f62 2031 3200 6865 6c6c 6f20 776f  blob 12.hello wo
00000010: 726c 640a                                rld.
shreeranganath@shreerangnath:~/Desktop/os-u4-orange-problem$
```

Phase 3: The Index (Staging Area) :

```
shreeranganath@shreerangnath:~/Desktop/os-u4-orange-problem$ ./pes status
Staged changes:
  staged:      file1.txt
  staged:      file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  commit.h
  untracked:  tree.c
  untracked:  a.txt
  untracked:  bye.txt
  untracked:  README.md
  untracked:  commit.c
  untracked:  Makefile
  untracked:  test_tree.c
  untracked:  index.h
  untracked:  .gitignore
  untracked:  index.c
  untracked:  hello.txt
  untracked:  test_objects.c
  untracked:  test_tree
  untracked:  object.c
  untracked:  pes.h
  untracked:  test_objects
  untracked:  pes.c
  untracked:  tree.h
  untracked:  test.txt
  untracked:  test_sequence.sh

shreeranganath@shreerangnath:~/Desktop/os-u4-orange-problem$
```

```
shreeranganath@shreerangnath:~/Desktop/os-u4-orange-problem$ cat .pes/index
100664 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776323980 6 file1.txt
100664 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776323988 6 file2.txt
shreeranganath@shreerangnath:~/Desktop/os-u4-orange-problem$
```

Phase 4: Commits and History :

```
shreeranganath@shreerangnath:~/Desktop/os-u4-orange-problem$ ./pes log
commit 1f86339d36a21ed142e02146efaef642a4b5fb1e01d66e0ee6b19a260681c3a1
Author: PES User <pes@localhost>
Date:   1776324354

    Add bye

commit 0eace1645103e95c7c4a8a8cee5a0dac9ca6c93c372c6a459fcd61855659d424
Author: PES User <pes@localhost>
Date:   1776324330

    Add world

commit 22433be079ccdd13054fdc9d59e46e7a9fbcc83ea6f079484a1c559455972b1e
Author: PES User <pes@localhost>
Date:   1776324307

    Initial commit

shreeranganath@shreerangnath:~/Desktop/os-u4-orange-problem$
```

```
shreeranganath@shreerangnath:~/Desktop/os-u4-orange-problem$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/0e/ace1645103e95c7c4a8a8cee5a0dac9ca6c93c372c6a459fcd61855659d424
.pes/objects/1f/86339d36a21ed142e02146efaef642a4b5fb1e01d66e0ee6b19a260681c3a1
.pes/objects/22/433be079ccdd13054fdc9d59e46e7a9fbcc83ea6f079484a1c559455972b1e
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
.pes/objects/48/1407f7237f7bd749dca31e2d8422bafe211741d5556f007e0faf821ffc7179
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/7d/c3b9d4cee22f645e501f8aaa822f6777b3198434adc74491632478799e9e8b
.pes/objects/92/19086282c09f0db9088484e1ef89f1b05ecfb420a8784a6de327022771fbec
.pes/objects/e0/0c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f
.pes/objects/e8/560050ee85f3a516e3aab4e54c712682fd0163e5b423d854146ba4e54067d9
.pes/objects/fe/bed41618fe89b49ea9b82684254a1e74aa7f80fdba38fc84b592ccca955267
.pes/refs/heads/main
shreeranganath@shreerangnath:~/Desktop/os-u4-orange-problem$
```

```
shreeranganath@shreerangnath:~/Desktop/os-u4-orange-problem$ cat .pes/refs/heads/main
1f86339d36a21ed142e02146efaef642a4b5fb1e01d66e0ee6b19a260681c3a1
shreeranganath@shreerangnath:~/Desktop/os-u4-orange-problem$ cat .pes/HEAD
ref: refs/heads/main
shreeranganath@shreerangnath:~/Desktop/os-u4-orange-problem$
```

Phase 5 & 6: Analysis-Only Questions :

Q5.1: Branch Checkout

A branch in Git is represented as a file containing the latest commit hash. To implement pes checkout <branch>, the system must first update the .pes/HEAD file to point to the selected branch in .pes/refs/heads/. Then, it must read the commit hash stored in that branch file and load the corresponding commit object. The working directory must be updated to match the tree structure of that commit by recreating files and directories from

the object store. The complexity arises because existing files in the working directory may need to be overwritten or deleted, and care must be taken to avoid losing uncommitted changes.

Q5.2: Dirty Working Directory

To detect a dirty working directory, the system compares the current files in the working directory with the entries stored in the index. For each tracked file, the hash of the current file is computed and compared with the hash stored in the index. If there is a mismatch, it indicates that the file has been modified but not staged. If such modified files also differ from the versions in the target branch, the checkout operation must be refused. This prevents accidental loss of uncommitted changes by ensuring that the working directory is clean before switching branches.

Q5.3: Detached HEAD

A detached HEAD occurs when the HEAD file points directly to a commit hash instead of a branch reference. In this state, any new commits are not associated with any branch and may become unreachable if no reference points to them. These commits still exist in the object store but are not easily accessible. To recover such commits, the user can create a new branch pointing to the commit hash. This restores access to those commits and prevents them from being lost during garbage collection.

Q6.1: Garbage Collection

To identify unreachable objects, the system performs a traversal starting from all branch references in .pes/refs/heads/. A data structure like a hash set is used to store all reachable object hashes. Beginning from each branch's latest commit, the system recursively visits all parent commits, associated tree objects, and blobs. After traversal, any object not present in the reachable set is considered unreachable and can be safely deleted. In a repository with 100,000 commits and 50 branches, most objects would still be reachable due to shared history, but the traversal may still involve visiting a large number of objects.

Q6.2: Garbage Collection Race Condition

Running garbage collection concurrently with a commit operation is dangerous because newly created objects may not yet be referenced by any branch. If garbage collection runs at the same time, it may mistakenly identify these objects as unreachable and delete them. This creates a race condition where valid data is lost. Git avoids this issue by using locking mechanisms and temporary references to protect newly created objects. It ensures that objects involved in ongoing operations are not deleted until they are safely linked to a branch.

Final Integration Test :

The complete PES-VCS system was tested using the provided integration test script. This test verifies all major functionalities including repository initialization, file staging, commit creation, commit history, and object storage. The system successfully created multiple commits and maintained the correct reference chain between HEAD and branch pointers. The object store was populated with blobs, trees, and commit objects as expected. All test cases were executed without errors, confirming that the implementation works correctly and integrates all components of the version control system

```
shreeranganath@shreerangnath:~/Desktop/os-u4-orange-problem$ make test-integration
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:      file.txt
  staged:      hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: ed01ae9644eb... Initial commit

Log after first commit:
commit ed01ae9644eb808ed38257c879e29a5209629ff11548866db3a403e5c165305c
Author: PES User <pes@localhost>
Date:   1776324573
```

```
    Initial commit


--- Second Commit ---
Committed: 239712f21f9c... Update file.txt

--- Third Commit ---
Committed: 8fcfcd13b263... Add farewell

--- Full History ---
commit 8fcfcd13b263595e8746798cdf753f486c0405cf09e9d43a979f645fd2ced7f4
Author: PES User <pes@localhost>
Date:   1776324573

    Add farewell

commit 239712f21f9c02b494f0e9ebd381cdcbac481a8ab2c9077a4daf90f33c28ff42
Author: PES User <pes@localhost>
Date:   1776324573

    Update file.txt

commit ed01ae9644eb808ed38257c879e29a5209629ff11548866db3a403e5c165305c
Author: PES User <pes@localhost>
Date:   1776324573

    Initial commit


--- Reference Chain ---
HEAD:
ref: refs/heads/main
```

```
    Update file.txt

commit ed01ae9644eb808ed38257c879e29a5209629ff11548866db3a403e5c165305c
Author: PES User <pes@localhost>
Date:   1776324573

    Initial commit


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
8fcfcd13b263595e8746798cdf753f486c0405cf09e9d43a979f645fd2ced7f4

--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/23/9712f21f9c02b494f0e9ebd381cdcbac481a8ab2c9077a4daf90f33c28ff42
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/84/c5fed72c2dd9a3130e2e9b98033827f62793fb80d0295ccedf5fc7972d96eb
.pes/objects/8f/cfcd13b263595e8746798cdf753f486c0405cf09e9d43a979f645fd2ced7f4
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff
.pes/objects/de/34fb483a5c16ce29ec3195dd505dfda7e35cc5fe16a99da4b2455b332a93cf
.pes/objects/ec/0af37b159ae92a5850ceaa6e497860729d6158aab2c6833b6068a1e3c5e8ef
.pes/objects/ed/01ae9644eb808ed38257c879e29a5209629ff11548866db3a403e5c165305c

=== All integration tests completed ===
shreeranganath@shreerangnath:~/Desktop/os-u4-orange-problem$
```